

Júlia Ribeiro de Souza

Desenvolvimento de Sistemas – Módulo 3 – Agenda 07

Atividade – Fichário – API no AppTempoAgora

Atividade feita com o auxílio da IA Google Gemini

Relatório das novas implementações no AppTempoAgora

Na parte 1 da atividade, foram adicionados novos campos no modelo **Tempo** para exibição no aplicativo: **Descrição visual do clima**, **Velocidade do vento** e **Visibilidade**. No arquivo **MainPage.xaml.cs**, utilizando a variável **t** do tipo **Tempo?**, foi feita a interpolação dessas propriedades no objeto para formatar os dados e exibi-los no rótulo **lbl_res.Text** para o usuário.

```
17     try
18     {
19         if(!string.IsNullOrEmpty(txt_cidade.Text))
20         {
21             Tempo? t = await DataService.GetPrevisao(txt_cidade.Text);
22
23             if(t != null)
24             {
25                 string dados_previsao = "";
26
27                 dados_previsao = $"Latitude: {t.Lat} \n" +
28                                 $"Longitude: {t.Lon} \n" +
29                                 $"Nascer do Sol: {t.Sunrise} \n" +
30                                 $"Pôr do Sol: {t.Sunset} \n" +
31                                 $"Temperatura Máxima: {t.TempMax}°C \n" +
32                                 $"Temperatura Mínima: {t.TempMin}°C \n" +
33                                 $"Velocidade do Vento: {t.Speed} m/s \n" +
34                                 $"Visibilidade: {t.Visibility} metros \n" +
35                                 $"Descrição do clima: {t.Description} \n";
36
37                 lbl_res.Text = dados_previsao;
38             }
39         }
40     }
```

Imagem 1 - Código dos novos dados implementados.

Home

Buscar

Latitude: -23,5017

Longitude: -47,4581

Nascer do Sol: 17/09/2026 06:04:44

Pôr do Sol: 17/09/2026 18:04:11

Temperatura Máxima: 17,63°C

Temperatura Mínima: 17,63°C

Velocidade do Vento: 5,69 m/s

Visibilidade: 10000 metros

Descrição do clima: overcast clouds

Na parte 2 da atividade, o foco foi tornar a aplicação mais robusta e amigável ao usuário através do tratamento de exceções e da verificação de respostas da API.

Funcionalidades Implementadas

- **Identificação de Cidade Não Encontrada:** Inclusão de verificação do código de status HTTP retornado pela API do OpenWeather. Quando a busca por um município inexistente retorna o código **HttpStatusCode.NotFound** (HTTP 404), uma exceção específica é lançada para alertar o usuário sobre o erro de digitação.
- **Deteção de Falha de Conexão (Offline):** Envolvimento da chamada de rede **HttpClient.GetAsync()** em um bloco **try...catch**. Ao capturar a exceção **HttpRequestException**, o sistema identifica que o dispositivo está sem acesso à internet ou não conseguiu alcançar o servidor, exibindo uma mensagem adequada.

```

18
19 using (HttpClient client = new HttpClient())
20 {
21     try {
22         HttpResponseMessage resp = await client.GetAsync(url);
23
24         if (resp.IsSuccessStatusCode)
25         {
26             string json = await resp.Content.ReadAsStringAsync();
27
28             var rascunho = JObject.Parse(json);
29
30             DateTime time = new(1970, 1, 1, 0, 0, 0, DateTimeKind.Utc);
31             DateTime sunrise = time.AddSeconds((double)rascunho["sys"]["sunrise"]).ToLocalTime();
32             DateTime sunset = time.AddSeconds((double)rascunho["sys"]["sunset"]).ToLocalTime();
33
34             t = new()
35             {
36                 Lat = (double)rascunho["coord"]["lat"],
37                 Lon = (double)rascunho["coord"]["lon"],
38                 Description = (string)rascunho["weather"][0]["description"],
39                 Main = (string)rascunho["weather"][0]["main"],
40                 TempMin = (double)rascunho["main"]["temp_min"],
41                 TempMax = (double)rascunho["main"]["temp_max"],
42                 Speed = (double)rascunho["wind"]["speed"],
43                 Visibility = (int)rascunho["visibility"],
44                 Sunrise = sunrise.ToString(),
45                 Sunset = sunset.ToString(),
46             }; //Fecha o objeto do tempo
47             // Fecha if se o status do servidor foi de sucesso
48             else if (resp.StatusCode == HttpStatusCode.NotFound)
49             {
50                 //Throw - usado para lançar na tela do usuário quando o sistema detecta um erro.
51                 throw new Exception("Cidade não encontrada. Verifique o nome digitado.");
52             }
53             else
54             {
55                 //Throw - usado para lançar na tela do usuário quando há um erro na API.
56                 throw new Exception("Erro na API: {resp.StatusCode}");
57             }
58         }
59         catch (HttpRequestException) //Ausência de conexão com a rede
60         {
61             throw new Exception("Sem conexão com a internet. Verifique sua rede e tente novamente.");
62         }
63     } // Fecha laço using
64
65     return t;
66 }

```

Imagem 3 - Código para tratamento de erros.

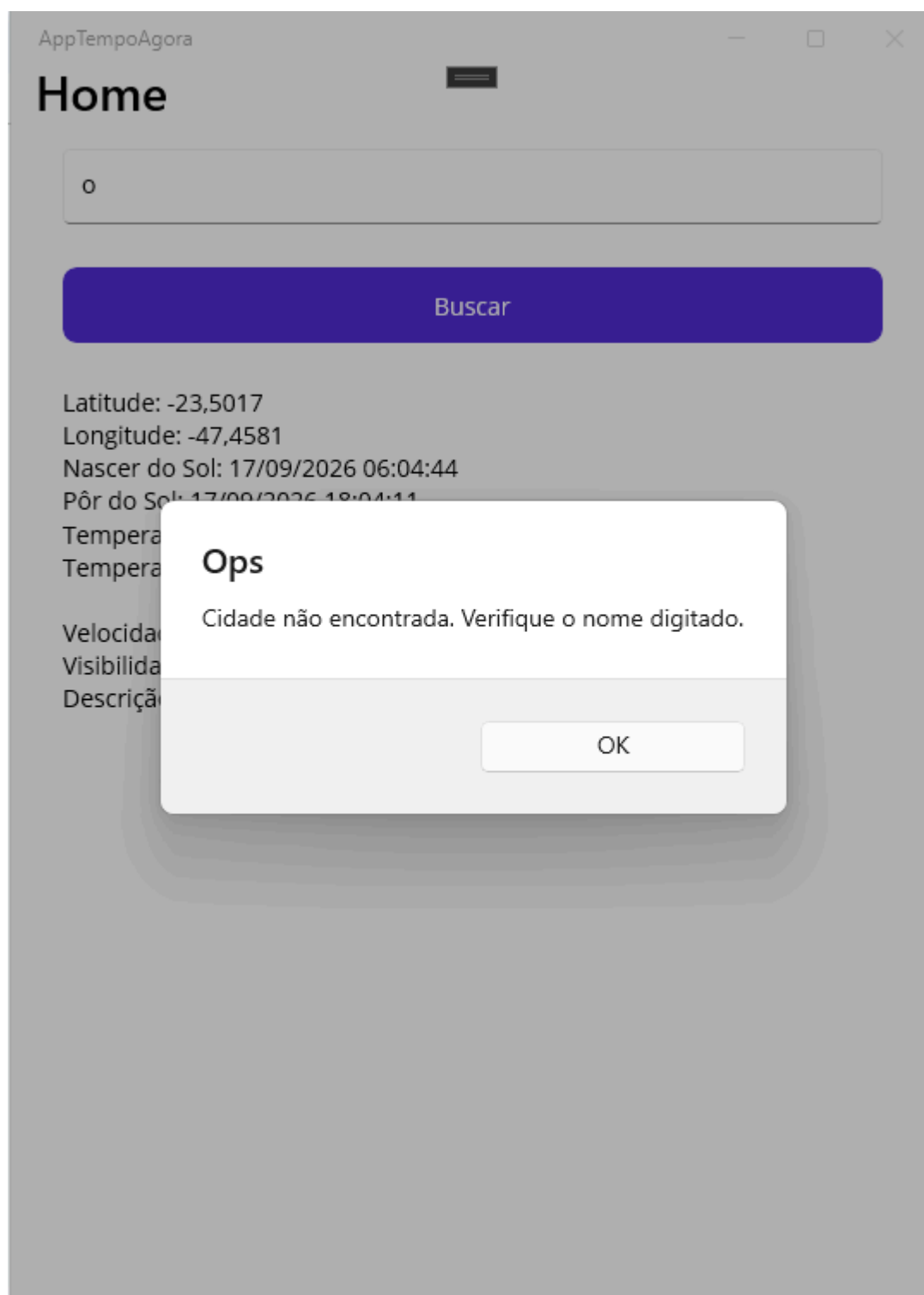


Imagem 4 - Execução do código. Erro na digitação da cidade.

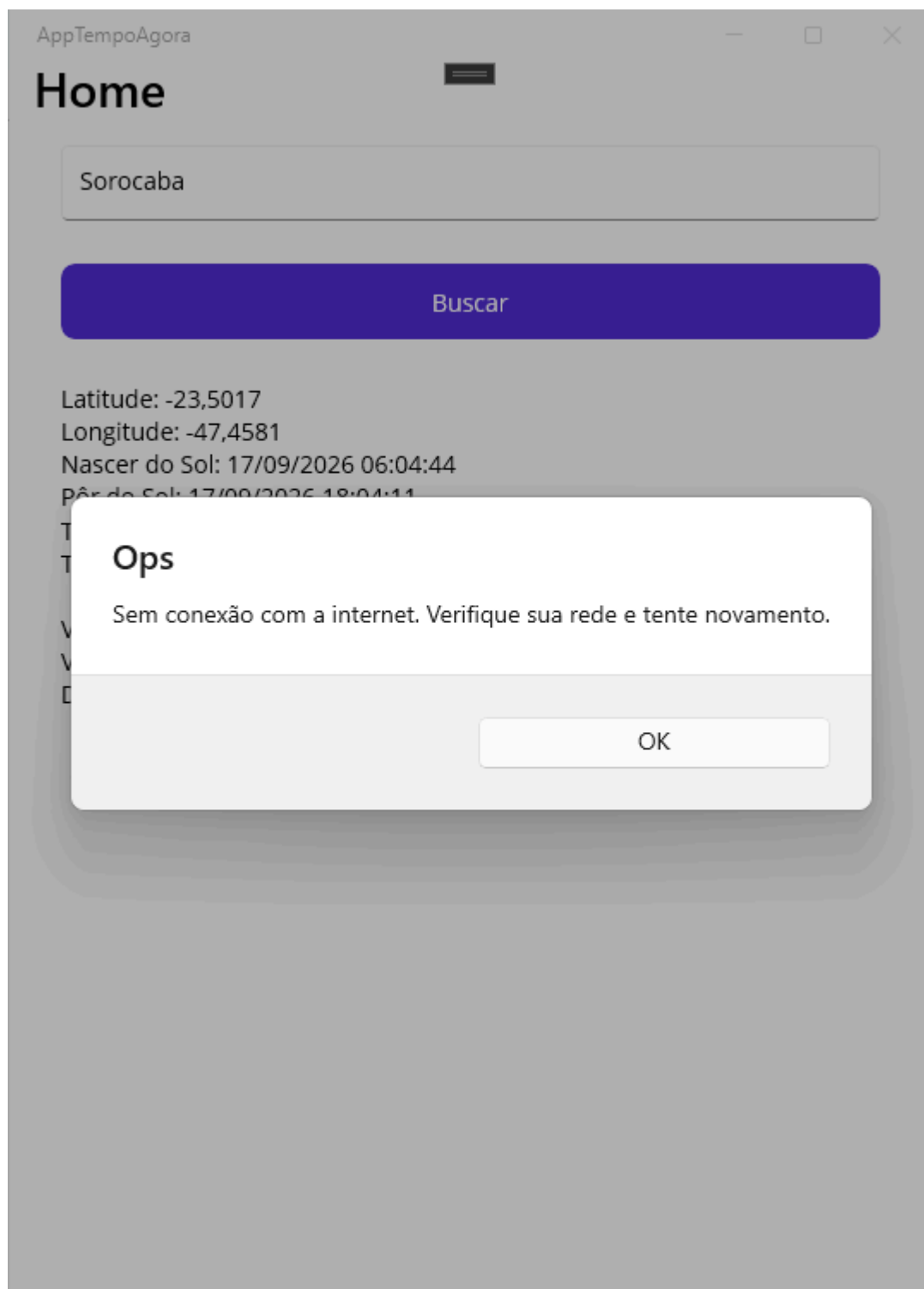


Imagem 5 - Execução do código. Erro na conexão com a rede.